

An Analysis of Backpropagation

Backpropagation

$$\frac{\partial L}{\partial W} = \delta \cdot a^T$$

$$\delta^{(l)} = (W^{(l+1)T} \delta^{(l+1)}) \odot \sigma'(z^{(l)})$$

Backpropagation -- Part 1

$$E = (t - y)^2 = y^2 = (x_1 w_1 + x_2 w_2)^2 = (w_1 + w_2)^2.$$

Backpropagation -- Part 2

$$\frac{dC}{d\alpha} \cdot W^{\circ}(fL)' \cdot W^{L-1} \cdots W^{f1}' \cdot W^1 \cdot W^{\circ}(fL-1) \cdots W^{f1}$$

Backpropagation -- Part 3

However, if j is in an arbitrary inner layer of the network, finding the derivative E with respect to o_j is less obvious. Considering E as a function with the inputs being all neurons $L = \{u, v, \dots, w\}$ receiving input from neuron j ,

Backpropagation -- Part 4

$$\delta_j = \frac{\partial E}{\partial o_j} = \begin{cases} (o_j - t_j) o_j (1 - o_j) & \text{if } j \text{ is an output neuron,} \\ \left(\sum_{\ell \in L} w_{\ell j} \delta_{\ell} \right) o_j (1 - o_j) & \text{if } j \text{ is an inner neuron.} \end{cases}$$

Backpropagation -- Part 5

$$\frac{\partial E(o_j)}{\partial o_j} = \frac{\partial E(\text{net}_u, \text{net}_v, \dots, \text{net}_w)}{\partial o_j}$$

Backpropagation -- Part 6

This is the reason why backpropagation requires that the activation function be differentiable. (Nevertheless, the ReLU

activation function, which is non-differentiable at 0, has become quite popular, e.g. in AlexNet) The first factor is straightforward to evaluate if the neuron is in the output layer, because then $o_j = y_j$ and

Backpropagation -- Part 7

The gradients of the weights can thus be computed using a few matrix multiplications for each level; this is backpropagation. Compared with naively computing forwards (using the $\delta 1$ for illustration):

Backpropagation -- Part 8

The input net j to a neuron is the weighted sum of outputs o_k of previous neurons. If the neuron is in the first layer after the input layer, the o_k of the input layer are simply the inputs x_k to the network. The number of input units to the neuron is n . The variable w_{kj} denotes the weight between neuron k of the previous layer and neuron j of the current layer.

Backpropagation -- Part 9

$$C(y, f_L(W_L f_{L-1}(W_{L-1} \cdots f_2(W_2 f_1(W_1 x)) \cdots))) \quad \{\displaystyle C(y, f^{\wedge\{L\}} f^{\wedge\{L-1\}}(W^{\wedge\{L-1\}} \cdots f^{\wedge\{2\}}(W^{\wedge\{2\}} f^{\wedge\{1\}}(W^{\wedge\{1\}} x)) \cdots))\}$$

Backpropagation -- Part 10

$$g(x) := f^L(W^L f^{L-1}(W^{L-1} \cdots f^1(W^1 x) \cdots))$$

Backpropagation -- Part 11

Note that δ_l is a vector, of length equal to the number of nodes in level l ; each component is interpreted as the "cost attributable to (the value of) that node". The gradient of the weights in layer l is then:

Backpropagation -- Part 12

y is the actual output of the output neuron. In this section, the order of the weight indexes are reversed relative to the prior section: w_{ij} is weight from the i th to the

Backpropagation -- Part 13

An Analysis of Backpropagation

Backpropagation

Modern backpropagation was first published by Seppo Linnainmaa as "reverse mode of automatic differentiation" (1970) for discrete connected networks of nested differentiable functions. In 1982, Paul Werbos applied backpropagation to MLPs in the way that has become standard. Werbos described how he developed backpropagation in an interview. In 1971, during his PhD work, he developed backpropagation to mathematicize Freud's "flow of psychic energy". He faced repeated difficulty in publishing the work, only managing in 1981. He also claimed that "the first practical application of back-propagation was for estimating a dynamic model to predict nationalism and social communications in 1974" by him. Around 1982, David E. Rumelhart independently developed backpropagation and taught the algorithm to others in his research circle. He did not cite previous work as he was unaware of them. He published the algorithm first in a 1985 paper, then in a 1986 Nature paper an experimental analysis of the technique. These papers became highly cited, contributed to the popularization of backpropagation, and coincided with the resurging research interest in neural networks during the 1980s. In 1985, the method was also described by David Parker. Yann LeCun proposed an alternative form of backpropagation for neural networks in his PhD thesis in 1987. Gradient descent took a considerable amount of time to reach acceptance. Some early objections were: there were no guarantees that gradient descent could reach a global minimum, only local minimum; neurons were "known" by physiologists as making discrete signals (0/1), not continuous ones, and with discrete signals, there is no gradient to take. See the interview with Geoffrey Hinton, who was awarded the 2024 Nobel Prize in Physics for his contributions to the field.

Backpropagation -- Part 14

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial net_j} \frac{\partial net_j}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial net_j} \frac{\partial net_j}{\partial w_{ij}}$$

Backpropagation -- Part 15

For a training set there will be a set of input-output pairs, $\{ (x_i, y_i) \}$. For each input-output pair (x_i, y_i) in the training set, the loss of the model on that pair is the cost of the difference between the predicted output $g(x_i)$ and the target output y_i :